

DATA 298B

MSDA Project II Workbook 1

Team 2

Kevin Gao, Andreah Cruz, Pukhraj Rathkanthiwar, Vrushabh Bodarya

October 5, 2026

Project: AI-Driven Persona Cloning for B2B Content Generation (Jason Lemkin and SaaStr)

4. Model Development

Our project builds a writing assistant that drafts LinkedIn posts, blog articles, X threads and talk scripts in the voice of Jason Lemkin, the founder of SaaStr. The assistant reads his own writing, finds the passages that matter for a given topic, and writes a draft that sounds like him and stays close to what he has actually said. The drafts are model output. They are not his words and he has not endorsed them.

Every model in this workbook is judged against the same five problems, so we name them once here and refer back to them by number.

- P1 Voice.** The draft should read like Lemkin: short paragraphs, direct advice, concrete examples.
- P2 Substance.** Claims in the draft should come from his writing, not from the model's imagination.
- P3 Medium.** A blog article, a LinkedIn post, an X thread and a talk have different shapes.
- P4 Completeness.** A draft should finish cleanly and not stop in the middle of a sentence.
- P5 Originality.** A draft should not copy long stretches from the source posts.

4.1 Model Proposals

We propose six models. Each one adds a single idea to the one before it, which lets us see what each idea is worth. Models A to E are built and have been run. Model F is a proposal that we have designed but not yet implemented, and we say so wherever it appears. Figure 1 shows how the models are put together.

Model A: base Llama 3.1 8B Instruct. This is the control. Llama 3.1 8B is a decoder-only transformer with grouped-query attention, trained by Meta and released with open weights [4]. We run it as it comes, through Ollama, with only the task description in the prompt. A model that has never seen our corpus tells us how much of the voice, the facts and the formatting come from the base model alone. It is the yardstick for everything else.

Model B: persona-prompted Llama. Model B keeps the same frozen model and adds a persona. We scanned all 31,443 documents in the corpus, about 2.7 million words, and wrote the results into a profile file: favourite vocabulary, sentence and paragraph structure, how he frames advice, and the themes he returns to. A short summary of that profile, together with the format specification for the chosen medium, is placed at the top of the prompt. No retrieval and no training are involved.

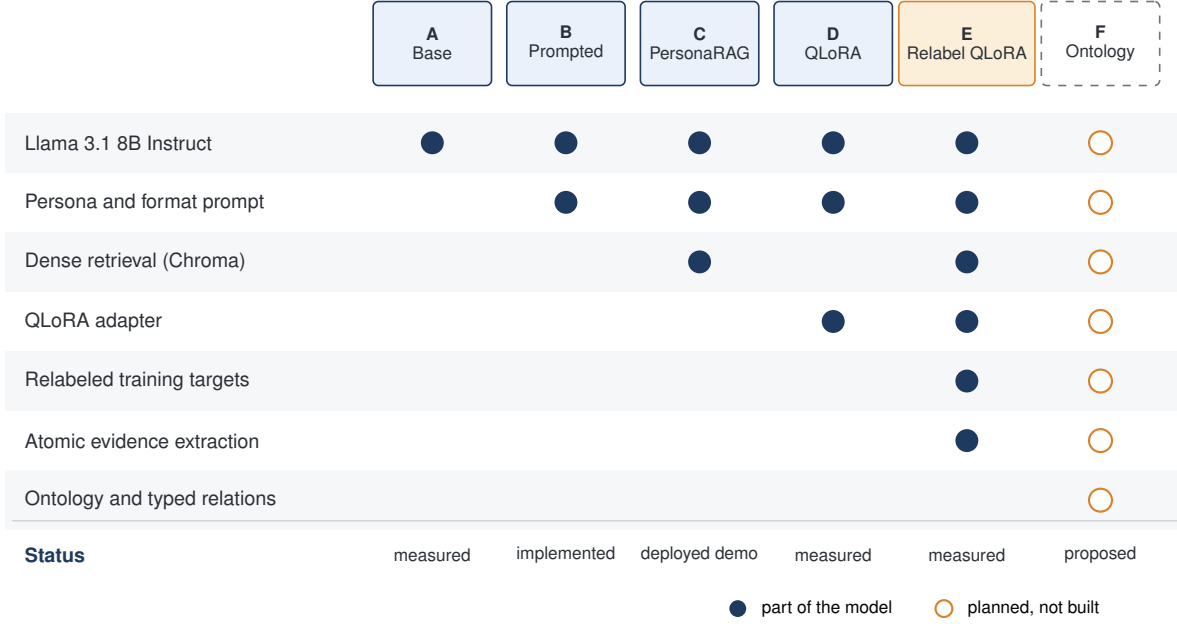


Figure 1. The six proposed models and the parts each one is built from. Filled circles are parts that exist in the model. Open circles belong to the proposed model F and are not built yet.

This addresses P1 and P3 with prompting alone. The profile is made of counts and frequencies, so this is the one place where the project uses plain statistics.

Model C: PersonaRAG, vector retrieval with persona. Model C adds a memory. The corpus is cut into chunks of about 500 words, and every chunk is turned into a 768-dimensional vector with the `nomic-embed-text` model [5]. The vectors live in a ChromaDB collection backed by an HNSW index [6], which finds the nearest vectors without comparing against every chunk. At question time we embed the topic, goal and audience as one query, fetch the closest chunks (between one and twenty, adjustable), and place them in the prompt next to the persona summary and the format specification. The generator is still the frozen base model. This follows the retrieval-augmented generation idea of Lewis et al. [1]. The name PersonaRAG comes from the published PersonaRAG work [7]. Our version is a simpler design of our own, and we do not claim it reproduces that paper. It targets P2, with help for P1 and P3.

Model D: QLoRA fine-tuned Llama. Model D changes the model itself. We freeze the 4-bit quantized base weights and train small low-rank adapters on top, which is the QLoRA recipe [3]. For a frozen weight matrix W the adapted layer computes $W + \frac{\alpha}{r}BA$, where A and B are the only trained matrices. We use rank $r = 16$, scaling $\alpha = 32$, no dropout, and attach adapters to seven projections in every block: query, key, value, output, gate, up and down. The model learns from instruction and answer pairs built from Lemkin’s own posts, with a 512-token window, a learning rate of 10^{-4} and a paged 8-bit AdamW optimizer. Loss is counted on the answer only. LoRA [2] keeps the trainable part small enough for a single 16 GB GPU. Model D addresses P1 and P3 directly, because the voice is learned and not described.

Table 1. Targeted problems addressed by each model. A tick means the model contains a part designed for that problem.

Model	P1 Voice	P2 Substance	P3 Medium	P4 Complete	P5 Original
A Base Llama 3.1 8B			✓		
B Persona-prompted	✓		✓		
C PersonaRAG	✓	✓	✓		
D QLoRA	✓		✓		
E Relabel QLoRA with factual RAG	✓	✓	✓	✓	✓
F Ontology hybrid (proposed)	✓	✓	✓	✓	✓

Model E: Relabel QLoRA with factual dense RAG and atomic evidence extraction. Model E is the improved model and our main proposal. It fixes three weaknesses we found in model D and in the usual way of combining fine-tuning with retrieval.

1. **Cleaner targets.** Our first fine-tunes often stopped in the middle of a sentence. When we audited the training file we found why. Long posts had been cut into chunks, and many chunks were still paired with an instruction that said “write the whole post”. The model learned to end early. We relabeled the rows as an opening, a continuation or an unchanged whole post, and dropped 3,685 rows we could not repair. Of 25,062 training rows, 21,377 remain.
2. **Facts kept apart from style.** The retrieval index is built from training documents only (21,193 of them), so no test or evaluation text can leak into what the model reads. Retrieved passages are not pasted into the writing prompt. They are used for facts, and the adapter supplies the style.
3. **Evidence check before writing.** With the adapter switched off, the base model turns the four retrieved passages into short claims, each with the exact words that support it. A plain program keeps a claim only if those words really appear in the named source. The writing step sees the accepted claims and nothing else. Afterwards we flag numbers that are not in the evidence and retry once if a draft copies twelve or more words in a row from a source.

Together these steps aim at all five problems, with P2, P4 and P5 as the main gains. Figure 3 in Section 4.2 shows the flow. This model is also the one the team has frozen as the deployment candidate.

Model F: ontology-aware hybrid (proposed). Model F is a design, not a result. Vector search finds passages that sound similar to a topic, but it cannot answer a question such as “what has Lemkin said about hiring a first sales rep, and did he recommend it or warn against it?” For that we would add a small ontology with six entity types (author, document, topic, entity, claim and stance) and typed relations between them, for example *document states claim* and *claim holds stance on topic*. Every claim would keep its source span so it stays traceable. Retrieval would follow those types and not only vector distance. Graph-based retrieval such as LightRAG [8] is one way to build it. Model F would reuse the generator of model E and be compared with plain vector retrieval on the same corpus, so any gain can be credited to the ontology. We list it so the model family is complete, and we mark it as planned.

Table 1 summarizes which problems each model is meant to address.

Table 2. Platform, tools and artifacts that support each model.

Model	Runs on	Main tools	Artifacts
A	Ollama on laptop, EC2 or GPU workstation	Llama 3.1 8B Instruct, Ollama	Model weights only
B	Same as A	Persona profile built by the Airflow DAG, format specifications in YAML, <code>generate.py</code>	<code>persona_profile.json</code>
C	Compose stack and EC2	Airflow, Spark, MinIO, ChromaDB, <code>nomic-embed-text</code> , FastAPI	Collection <code>lemkin_content</code>
D	Training on the GPU workstation, serving through Ollama	Unsloth, TRL, PEFT, 4-bit Llama 3.1 8B, <code>llama.cpp</code> for GGUF export	LoRA adapter, GGUF file
E	GPU workstation, loaded with Unsloth in one process	Unsloth, ChromaDB, <code>nomic-embed-text</code> , evidence and copy-check modules	Relabel adapter, collection <code>lemkin_train_only</code>
F	Not built	Would add a graph or typed store in front of the generator of E	None yet

4.2 Model Supports

The models run in three environments, and each has a different job.

A laptop with Docker Compose runs the data pipeline and the application stack. Airflow 2.8.1 schedules the work, Spark runs in local mode inside the Airflow container, MinIO stores files, ChromaDB 1.5.5 stores vectors, PostgreSQL keeps Airflow’s own records, and Ollama on the host machine serves the embedding model. This is where the corpus becomes chunks and embeddings.

A GPU workstation trains and tests the adapters. It runs Python 3.11 with PyTorch 2.10 for CUDA 12.8, Unsloth, TRL and PEFT, on a 16 GB NVIDIA Blackwell GPU. A training run of the relabel adapter took about 3.4 hours.

One AWS EC2 instance hosts the shared demo. It runs the same Compose stack with a web gateway in front, and an optional GPU variant for faster generation. Section 5.2.2 describes it.

Table 2 lists, for each model, where it runs and what it needs.

Two data flows matter most. Figure 2 shows how training data is prepared and how an adapter is trained and scored. Figure 3 shows how model E turns a content brief into a checked draft.

Export lesson. Our first fine-tuned model produced gibberish once it was served in Ollama, even though the adapter itself was fine. The export path had dequantized the 4-bit weights wrongly. We now merge the adapter into the full-precision base model on the CPU, convert the result with `llama.cpp`, and quantize it to the Q4_K_M GGUF format that Ollama loads. Every scored run also saves its Git commit, dataset hashes, adapter path and embedding model next to the outputs, so a number can always be traced back to the exact setup that produced it.

4.3 Model Comparison and Justification

Table 3 compares the six models on the points the course asks for: the problems they target, the kind of method they use, the data they need, and their strengths and limits. By approach, A, D and E are deep learning models trained or released as neural networks. B adds descriptive statistics, the counts and frequencies in the persona profile. C adds machine learning in the form of learned embeddings and approximate nearest-neighbour search. F would add a knowledge graph.

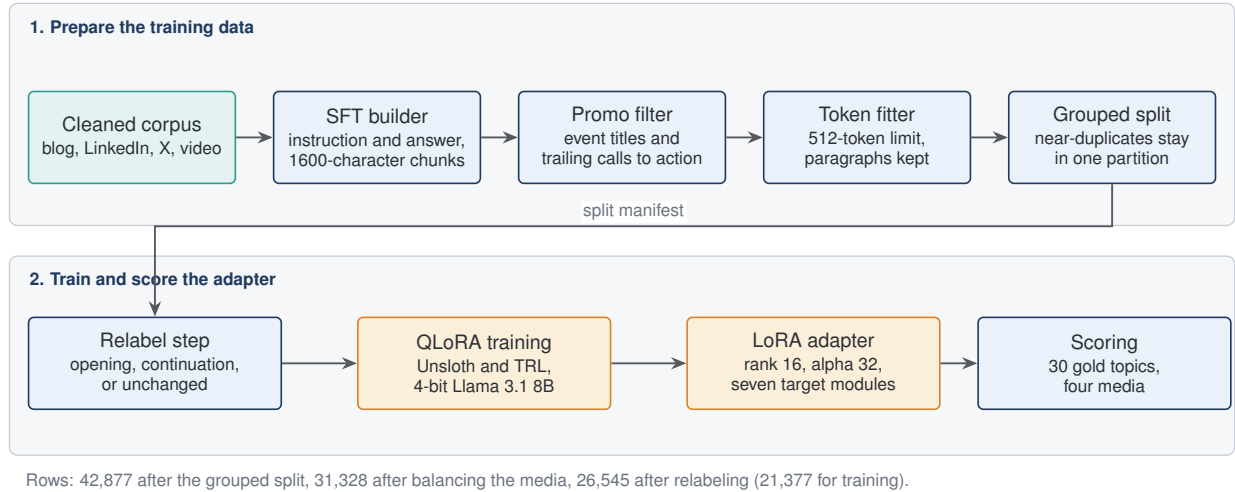


Figure 2. Training data flow. Rows are filtered, fitted to 512 tokens, split so that near-duplicates stay together, relabeled, and then used to train the adapter that is scored on the gold topics.

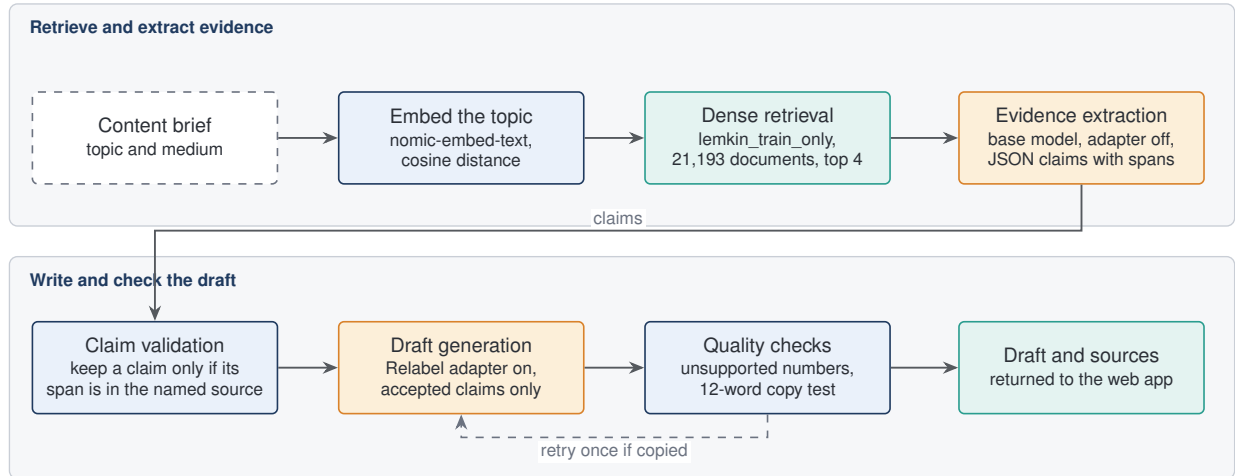


Figure 3. Inference data flow of model E. Retrieved passages become validated claims, and only those claims reach the writing step.

Table 3. Comparison of the six models.

Model	Approach and data	Problems	Strengths	Limitations
A	Deep learning, pretrained model, no corpus	P3	Simple, nothing to train or host beyond Ollama. Follows format instructions well.	Knows nothing specific about Lemkin. Voice and facts come only from general training.
B	Deep learning plus statistics, persona profile from the corpus	P1, P3	Cheap and fast to change. No GPU training.	A description of style is a weak teacher. Cannot supply facts.
C	Deep learning plus machine learning, 38,105 embedded chunks	P2, P1, P3	Draft is tied to real passages. The corpus can be updated without retraining.	Retrieved text can be copied or misread. Voice still depends on the base model.

Model	Approach and data	Problems	Strengths	Limitations
D	Deep learning, fine-tuned on instruction pairs from the cleaned corpus	P1, P3	Voice is learned and not described. Small adapter, easy to swap.	Stops early when training targets are cut. No grounding. Can state facts that are wrong.
E	Deep learning plus machine learning, relabeled pairs and a train-only index of 21,193 documents	P1 to P5	Facts are traceable to source spans. Fewer mid-sentence stops. Leak-safe index.	More steps and more latency. Still produces some numbers that are not in the evidence. Gains in voice are not proven.
F	Deep learning plus knowledge graph, typed claims and relations	P2, P1	Could answer structured questions that vector search cannot.	Not built. Needs a schema, extraction quality, and source attribution we do not yet have.

Table 4 compares the non-functional side. The ratings are relative to each other and not measured values.

Why each model is in the set. **A** is required for any honest comparison, because without it we could not say whether anything we built helps. **B** is the cheapest way to try to get the voice, and the right test of whether the heavier models earn their cost. **C** is the standard way to bring facts to a language model, and it keeps the corpus editable. **D** tests whether the voice can be learned and not just described, which is the central question of the project. **E** exists because **D** and **C** each fail in a way the other does not fix: **D** invents facts and **C** does not change the voice. **E** combines them with stricter data and a check on the evidence. **F** answers a requirement raised for the course, an ontology-aware retrieval design, and it is the natural next step once **E** is stable.

Alternatives we considered. *Keyword retrieval with BM25* [9] is a statistical method and would be the fairest baseline for model **C**. We did not build it for this workbook, and we plan to add it to the retrieval comparison. *Hosted fine-tuning* through the OpenAI service was prepared by one

Table 4. Non-functional comparison. Ratings are relative within this table.

Model	Hardware to serve	Training needed	Moving parts	Traceability
A	One 8B model	None	The model	None
B	One 8B model	None	Model, persona profile	Low
C	One 8B model plus embeddings	None	Chroma, Ollama, API, profile	Medium, sources shown
D	One 8B model plus adapter	Hours on a 16 GB GPU	Adapter, export, Ollama	None
E	One 4-bit model with the adapter switched on and off, plus embeddings	Hours on a 16 GB GPU	Chroma, adapter, extraction, validator, checks	High, claim to source span
F	As E plus a graph store	As E	As E plus the graph and its schema	Highest, typed provenance

teammate, with a dataset builder and a job script for a small model, as a side path. We kept the main line on open weights because we can inspect them, run them on our own hardware, and control cost. *Full fine-tuning* of all 8 billion weights does not fit on a 16 GB GPU, which is why we use QLoRA.

4.4 Model Evaluation Methods

Judging a writing model is harder than judging a classifier, because there is no single correct answer. We evaluate on several separate axes and refuse to average them into one score. The reason is practical. When we first used a single blended score, a model that stopped in the middle of a sentence could still look good if its word overlap with the reference was high.

4.4.1 Evaluation data and protocol

The gold panel has 30 topics, each written for four media (blog, LinkedIn, X and talk), which gives 120 prompts per model. The training data uses an 80, 10 and 10 split between training, validation and test. Rows that are near-duplicates of each other, by title, by exact text or by a word-overlap score of 0.8 or more, are kept in the same partition so that a post and its copy cannot land on opposite sides. Generation settings, prompts, seeds and the scoring code are frozen before a run. The gold topics have been used during development, so we call them a development panel and not an untouched test set.

4.4.2 Metrics

Table 5 lists each metric, what it measures and how it is computed.

Table 5. Evaluation metrics.

Metric	What it measures	How it is computed
Training and validation loss	How well an adapter fits its data (D, E)	Cross-entropy on the answer tokens, per evaluation checkpoint
Completion (P4)	Whether a draft finishes	Share of drafts that end early, stop mid-sentence or hit the length limit
Copying (P5)	Memorization of sources	Longest run of words shared with a source, excluding words in the prompt. Twelve or more fails
Persona style (P1)	Closeness to Lemkin’s surface habits	Mean absolute z-score of sentence length, questions, contractions, first person, numbers, function words, punctuation, type-token ratio and paragraphing, against held-out posts of the same medium
Perspective fidelity (P1)	Whether advice matches his stance	Share of advice claims that agree with held-out Lemkin evidence

Metric	What it measures	How it is computed
Writing quality (P1)	Coherence, fluency, logic, relevance, specificity, usefulness	Blinded judge model scores each from 1 to 5
Task adherence (P3)	Topic and medium fit	Topic-word coverage plus a length band for the medium
Grounding (P2)	Whether claims are supported	Each claim marked supported, unsupported, contradicted or not verifiable against the retrieved passages
Numeric check (P2)	Invented figures	Numbers in the draft that do not occur in the accepted evidence
Retrieval diagnostics	Behaviour of the retriever	Empty-result rate, duplicate rate, mean cosine distance, source mix
Reference similarity	Overlap with a constructed reference	BERTScore [10], BLEU, ROUGE-L. Reported only as diagnostics
Gold rubric	Agreement with the brief	Legacy five-part word-overlap average over format, brief, facts and rubric points. Reported but not used to choose a model

For persona style, with F surface features, a draft’s value x_f , and the mean μ_f and standard deviation σ_f of held-out posts of the same medium, the distance is

$$D_{\text{style}} = \frac{1}{F} \sum_{f=1}^F \frac{|x_f - \mu_f|}{\sigma_f}. \quad (1)$$

A smaller value means the draft is closer to how Lemkin actually writes in that medium. The grounding rate is the number of supported claims divided by the number of verifiable claims.

4.4.3 Selection rule

A draft is eligible only if it finishes, passes the copying limit and fits the medium, and, for RAG models, passes the grounding check. Style cannot average away a truncated or copied draft. Among eligible drafts we compare, in this order, persona style, perspective fidelity, writing quality and task adherence. We also report a combined Persona Utility score,

$$U = 0.45 S_{\text{style}} + 0.20 S_{\text{perspective}} + 0.25 S_{\text{writing}} + 0.10 S_{\text{task}}, \quad (2)$$

where each S is the calibrated score of that component. The weights are a project decision. The writing judge is a different model family from the Llama candidates (Qwen 2.5 14B, run through Ollama), so a model is not grading its own relatives [11]. BLEU, ROUGE-L, BERTScore and the gold rubric are never used to choose a model.

4.4.4 Metrics used for each model

Table 6 shows which metrics apply to which model.

Table 6. Metrics applied to each model. A tick means the metric is defined for that model.

Metric	A	B	C	D	E	F
Training and validation loss				✓	✓	✓
Completion, copying, task adherence	✓	✓	✓	✓	✓	✓
Persona style, perspective, writing quality	✓	✓	✓	✓	✓	✓
Grounding and numeric check			✓		✓	✓
Retrieval diagnostics			✓		✓	✓
Gold rubric and reference similarity	✓	✓	✓	✓	✓	✓

Table 7. Training of the relabel adapter (model E).

Setting	Value
Base model	Meta-Llama-3.1-8B-Instruct, 4-bit
Adapter	Rank 16, alpha 32, dropout 0, seven projection modules
Data	21,377 training, 2,588 validation, 2,580 test rows
Schedule	2 epochs, batch 1, gradient accumulation 16, learning rate 10^{-4} , seed 42
Sequence length	512 tokens
Run time	About 3.4 hours
Final training loss	1.797
Best validation loss	1.3316, at step 2,674

4.4.5 Known weaknesses of the metrics

We want to be clear about what the metrics cannot do. The gold rubric rewards headings, lists and word overlap, so it favours a model that follows a template over one that sounds like a person. Surface style counts are a rough proxy for voice and do not prove authorship. The judge model is a second opinion and not ground truth. Blinded human preference sheets are prepared but have not yet been used to choose a model. The 30 gold topics are few, and a single-seed gap should not be read as a settled ranking.

4.5 Model Validation and Evaluation Results

This section reports what has actually been measured. Models A, D and E have registered scores. Models B and C run in the application and the scripts, but they do not yet have scores on the same 120 prompts, and scoring them is the main experiment still to do. Model F is not built. All numbers come from registered experiment folders in our repository.

4.5.1 Training

Table 7 gives the settings and outcome for the relabel adapter, and Figure 4 shows its validation loss.

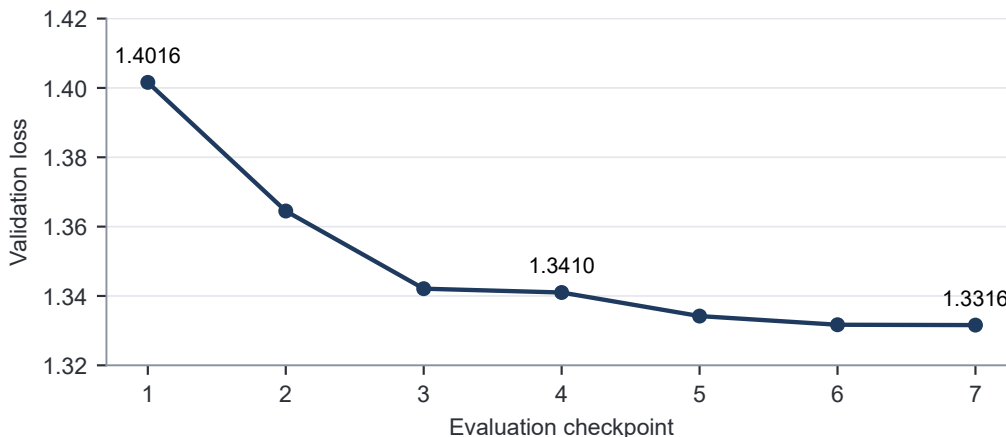


Figure 4. Validation loss of the relabel adapter over seven evaluations. The curve flattens after the fifth. This loss cannot be compared with the earlier repaired adapter (1.5311), because the validation prompts changed with the relabeling.

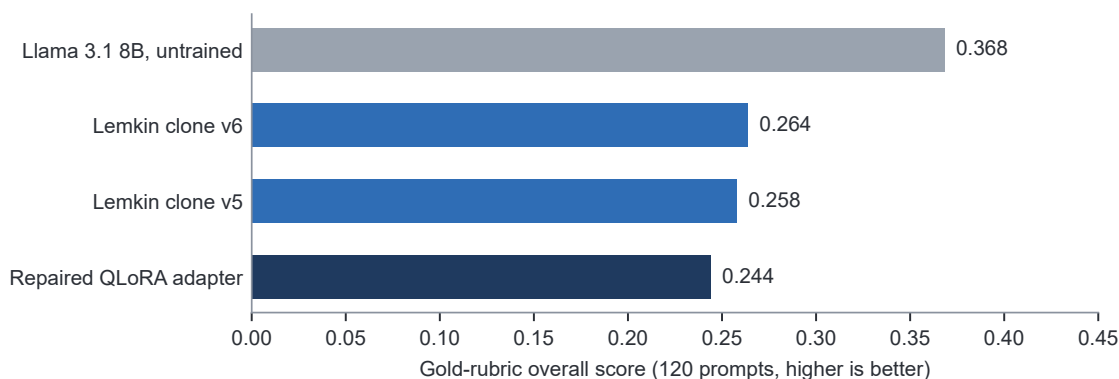


Figure 5. Gold-rubric overall score on 120 prompts. The untrained model leads because the rubric rewards format. The score is shown for completeness and is not used to select a model.

4.5.2 Untrained model against the fine-tunes

On 120 prompts, we scored the untrained Llama 3.1 and three fine-tuned variants with the legacy gold rubric (Figure 5). The untrained model scored highest, 0.368 against 0.244 for the repaired adapter. We did not expect that, and it taught us something. The rubric rewards headings, lists and overlap with a reference written by us, and an instruction-tuned model follows that template better than a model trained on Lemkin’s plainer posts. BLEU and ROUGE-L stayed near 0.004 and 0.08 for every model. This is why we stopped choosing models by this score and built the separate axes of Section 4.4. The two Lemkin clone results come from the same weights and show the gap between two samples.

4.5.3 Does relabeling fix the early stopping?

The relabel adapter stops cleanly more often than the repaired adapter (Figure 6). Mid-sentence stops fall from 63% to 34% of drafts, and early end-of-text from 65% to 55%. The gold-rubric score barely moves (0.271 against 0.273), which is what we want from a fix that targets completion and not format. We repeated the comparison with two more seeds. Across all three seeds the relabel adapter stopped mid-sentence in 34%, 30% and 33% of drafts, against 63%, 60% and 64% for the

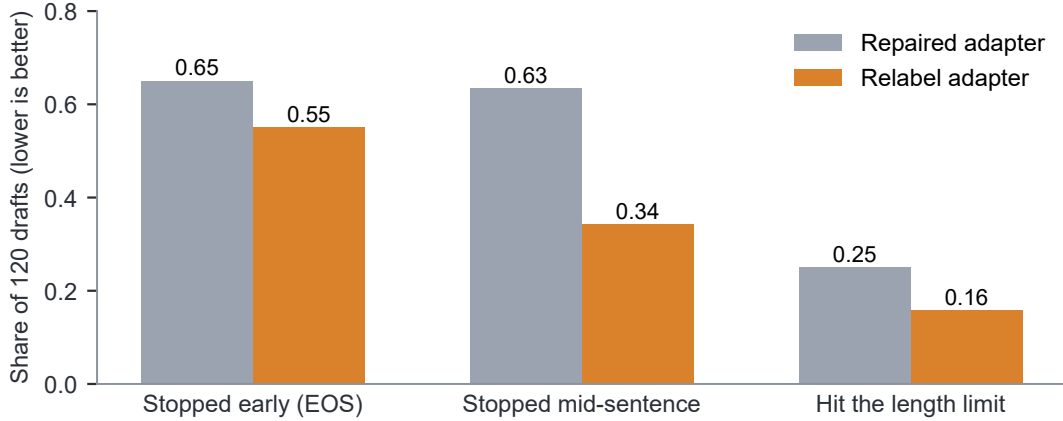


Figure 6. Stopping behaviour of the repaired and relabel adapters on 120 drafts. Lower is better.

Table 8. Retrieval diagnostics for the train-only index on the 30 gold topics, top four results.

Measure	Value
Documents in the index	21,193
Queries with no result	0 of 30
Duplicate document ids in a result list	0
Queries with two hits from one source group	11 of 30
Mean cosine distance (median)	0.243 (0.238)
Retrieved slots: blog, X, LinkedIn, YouTube	69, 39, 12, 0

repaired adapter, so the gain is not a lucky draw.

4.5.4 Retrieval

Before judging any generator, we looked at what the retriever returns for the 30 gold topics (Table 8). No query came back empty and no document appeared twice. The mean cosine distance was 0.243. Blog passages take 57% of the retrieved slots even though X posts are more than half of the index (Figure 7), which suggests that longer posts match topic queries better than short ones. For 11 of 30 queries, two of the four hits came from the same source group, so a diversity step is worth trying. We did not judge relevance by hand yet, so we report no precision or recall.

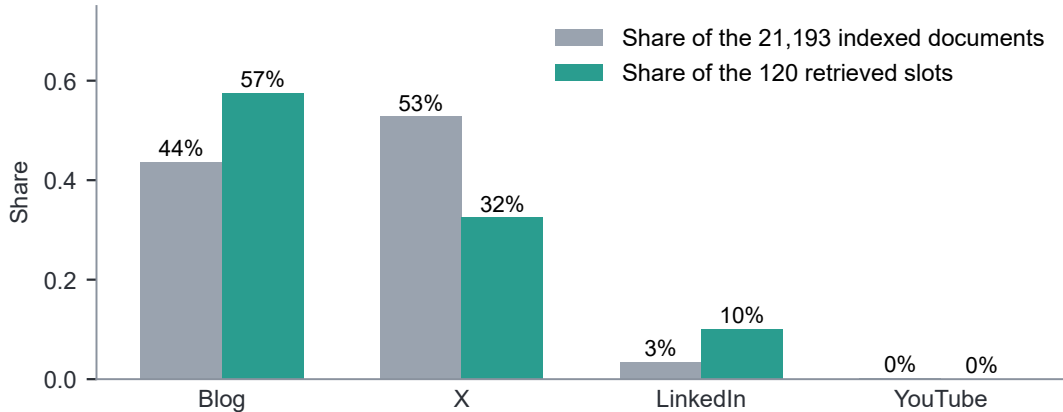


Figure 7. Platform mix of the index compared with what the retriever returns.

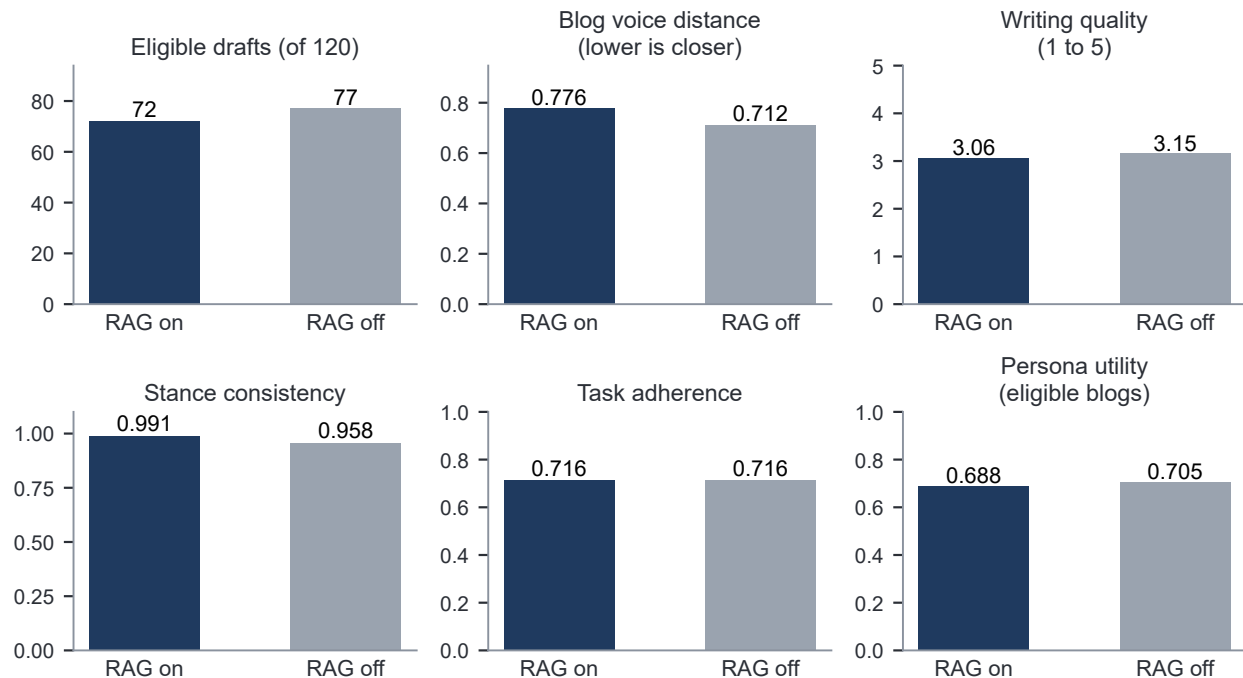


Figure 8. Paired comparison of the relabel adapter with retrieval on and off.

4.5.5 Retrieval on and off for the final candidate

The most direct test of model E’s design is a paired run, the same adapter and the same prompts with retrieval and evidence extraction on, and again with them off (Figure 8). All 120 drafts exist for both settings.

The result is mixed, and we report it as it is. Retrieval did *not* improve voice or writing quality. Blog voice distance went from 0.712 to 0.776 and writing quality from 3.15 to 3.06, and fewer drafts were eligible, 72 against 77. Persona utility on the eligible blogs was slightly lower, 0.688 against 0.705. Task adherence was equal, and stance consistency rose slightly, from 0.958 to 0.991. What retrieval did add is grounding. Of 600 claims examined, 498 could be checked against the evidence and 495 of those were supported, a rate of 99.4%. That is why we keep retrieval switched on, with its limits stated. We do not claim it makes the writing better.

4.5.6 A sample, checked by hand

To validate the numbers with a real case, we followed one draft from start to end. The prompt asked for an X post for B2B SaaS founders on whether to use an AI sales-development rep before a human one works. The retriever returned four passages (one blog, one LinkedIn and two X posts, cosine distances 0.194 to 0.245). The extraction step proposed 14 claims, the program rejected 10 because their supporting words were not in the named source, and 4 survived. Those four claims were the only facts the writer saw. The draft passed the copying test, with a longest shared span of three words.

Model E output (excerpt). Generated text, not written by Jason Lemkin.

Accepted evidence: 95% of human SDRs do not know the product they sell. 95% of human SDRs cannot answer tough questions. 95% of human SDRs have to wait for help. AI SDRs are much better than human SDRs.

Draft: Dear SaaS: Should I Use an AI SDR Before I Have a Working Human SDR motion?

Yes. The reality today is you will get so many more emails with an AI SDR that can work around the clock (with some limitations) than any human could ever manage.

AI can be much better than humans at doing prospecting and qualification before someone wants to buy. *[two short paragraphs follow]*

The sample shows the design working, since every fact traces to a retrieved passage. It also shows a limit. The draft reads as a plausible, short answer and does not yet carry much of his personal detail.

4.5.7 Where the candidate still falls short

Our checks of the saved traces found several weaknesses.

- 14 of 120 drafts fail the grounding gate. Twelve have no verifiable claim and two contain a contradicted claim.
- 60 of 120 drafts contain a number that is not in the accepted evidence. These rows are flagged and still returned as drafts.
- Two drafts copy a retrieved passage for more than twelve words.
- 17 drafts ended because they used the token budget and not because the model finished.
- Short media suffer most. By our own count of the saved traces, 20 of the 30 X drafts begin with the prompt title and run under 40 words in total, and on LinkedIn it is 6 of 30. Most of these are little more than the title. Blog and talk drafts were not affected. This is the biggest open problem, and it matters for the demo.

These points guide the next round. We plan to score models B and C on the same 120 prompts, add a keyword baseline to the retrieval comparison, review relevance by hand, and rework how short media are prompted.

5. Data Analytics System

5.1 System Requirements Analysis

5.1.1 System boundary, actors and use cases

The system boundary in Figure 9 contains the web app, the API, the data pipeline, the retrieval and generation services, and the training and evaluation tools. Outside it sit the people who use it and the services it depends on, namely Ollama, the Hugging Face model hub and the judge model. We have four actors.

- The **content creator** writes a brief, picks a medium and a method, and uses the draft.
- The **reviewer** checks the retrieved sources and judges whether a draft is faithful.
- The **ML engineer** fine-tunes adapters and scores models.
- The **platform operator** runs the pipeline and watches the health of the services.

Table 9 describes each use case.

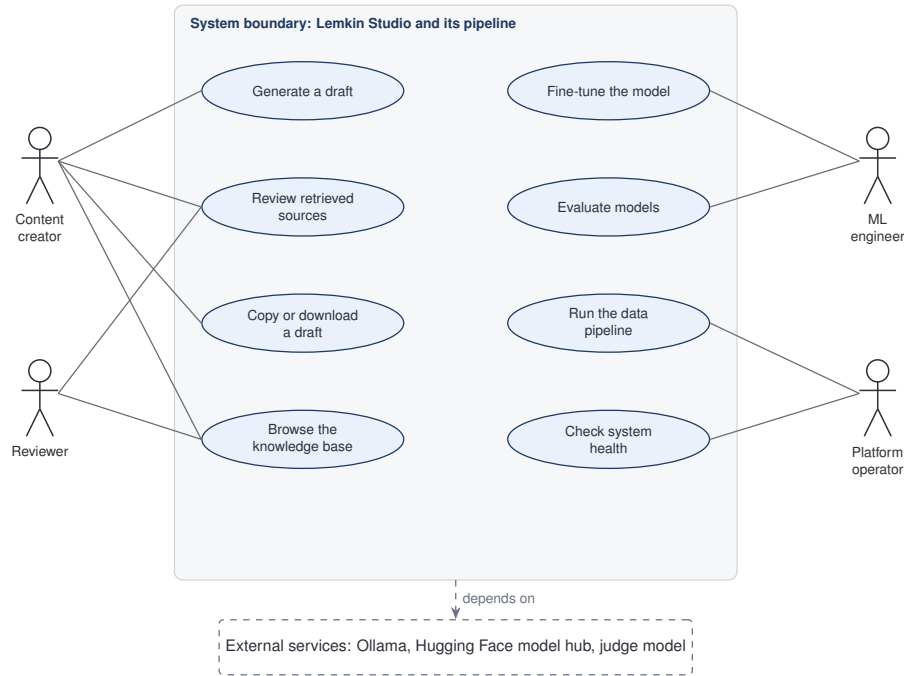


Figure 9. System boundary, actors and use cases.

Table 9. Use cases.

ID	Use case	Actor	What happens
UC1	Generate a draft	Content creator	The user enters topic, audience, goal and an optional call to action, picks a medium and a method, and receives a draft with its sources.
UC2	Review retrieved sources	Content creator, reviewer	The user reads the passages behind the draft and their similarity scores, to judge whether the draft is grounded.
UC3	Copy or download a draft	Content creator	The user copies the text or saves it as a Markdown file.
UC4	Browse the knowledge base	Content creator, reviewer	The user searches the corpus and filters by blog, LinkedIn, X or YouTube.
UC5	Fine-tune the model	ML engineer	The engineer builds the training file, trains an adapter and registers the run.
UC6	Evaluate models	ML engineer	The engineer scores saved drafts on the gold panel and reads the comparison.
UC7	Run the data pipeline	Platform operator	The operator triggers ingestion, cleaning, embedding and loading, and checks the counts.
UC8	Check system health	Platform operator	The operator sees whether the API, Ollama, ChromaDB and both models are ready.

5.1.2 Functional and non-functional requirements

Table 10 lists the requirements we design to.

Table 10. Requirements.

ID	Requirement
FR1	The user can choose among four media: LinkedIn post, blog article, X thread and YouTube script.
FR2	The user supplies a topic, an audience and a goal, and may add a call to action.
FR3	The system retrieves a chosen number of source passages, from 1 to 20.
FR4	The user can pick the writing method, PersonaRAG or QLoRA.
FR5	The system returns the draft with sources, model name, latency and a request id.
FR6	The user can copy the draft or download it as Markdown.
FR7	The system reports the health of the API, Ollama, ChromaDB and the models.
FR8	An operator can run ingestion, cleaning, embedding and training export end to end.
FR9	An engineer can train an adapter and score it on the gold panel, with the run saved and traceable.
NFR1	Traceability. Every claim shown to the writer links to a source passage. Every scored run records its Git commit, dataset hashes and model files.
NFR2	Reliability. Failures return a clear message with a hint for fixing it. External calls retry. The Python code has more than 100 automated tests and the web app has its own test suite.
NFR3	Safety of data. No test or evaluation text may enter the retrieval index used for generation.
NFR4	Security and privacy. Secrets stay out of the repository. The cloud demo is reachable only through an SSH tunnel or a password gateway. The web app keeps recent drafts in the user’s own browser.
NFR5	Cost. The cloud demo runs under a \$20 spend limit with alerts at \$5, \$10 and \$15.
NFR6	Responsible use. Output must be labeled as machine generated and never presented as the author’s own words.

5.1.3 Data analytics and machine learning capabilities

Table 11 lists the high-level functions the system provides. Each one names its input, its output and the technique behind it.

Table 11. Data analytics and machine learning capabilities.

Capability	Input	Output	Technique
Corpus analytics	Five JSONL source files	Counts, cleaned documents, persona profile	Parsing, filtering, frequency statistics
Chunking and embedding	Cleaned documents	38,105 vectors of 768 dimensions	Spark, <code>nomic-embed-text</code>
Semantic retrieval	Topic, goal, audience	Top- <i>k</i> passages with distances	Dense vectors, HNSW nearest-neighbour search
Draft generation	Brief, persona, passages	A draft in the chosen medium	Llama 3.1 8B, optionally with a QLoRA adapter
Evidence extraction and validation	Retrieved passages	Claims with exact source spans	Prompted extraction, then string-match validation
Output checking	A draft	Pass or flag for copying, numbers, finish	Rule-based checks
Model evaluation	Saved drafts and gold panel	Scores on separate axes	Style statistics, judge model, claim-level grounding
Monitoring	Service endpoints	Health state of each service	Dependency checks

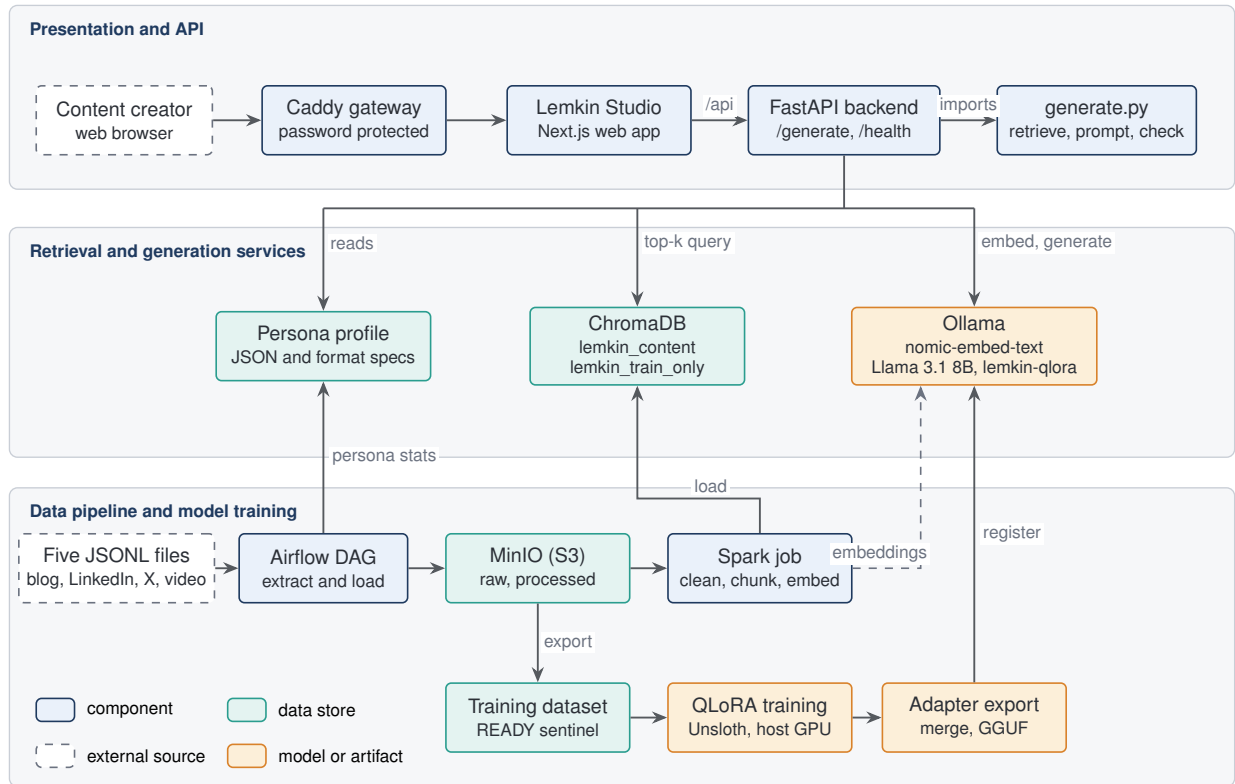


Figure 10. System architecture. Blue boxes are application components, green boxes are data stores, orange boxes are models or model artifacts, and dashed boxes are external.

5.2 System Design

5.2.1 System architecture and infrastructure

Figure 10 shows the architecture in three layers. The top layer is what a user touches, a password-protected gateway in front of a Next.js web app that calls a FastAPI service. The middle layer holds the services that do the AI work, ChromaDB for retrieval and Ollama for embeddings and generation, together with the persona profile and the format specifications. The bottom layer is the data pipeline and the training path.

The AI-powered function components are these.

- **Embedding service.** Ollama serves `nomic-embed-text`. It is used by the Spark job to embed chunks and by the API to embed each query.
- **Retrieval service.** ChromaDB answers nearest-neighbour queries over the two collections.
- **Generation service.** Ollama serves the base model and the fine-tuned model. For model E the adapter is switched on and off inside one loaded model.
- **Evidence and check modules.** Python modules extract claims, validate their spans, and flag copied text and unsupported numbers.
- **API layer.** FastAPI exposes the generation and health calls to the web app.

At this point in the project the live demo serves two methods, PersonaRAG with the base model and QLoRA through an Ollama export of a fine-tuned adapter. Model E is the frozen candidate, it runs in a single Python process, and wrapping it as a service behind the same API is the next

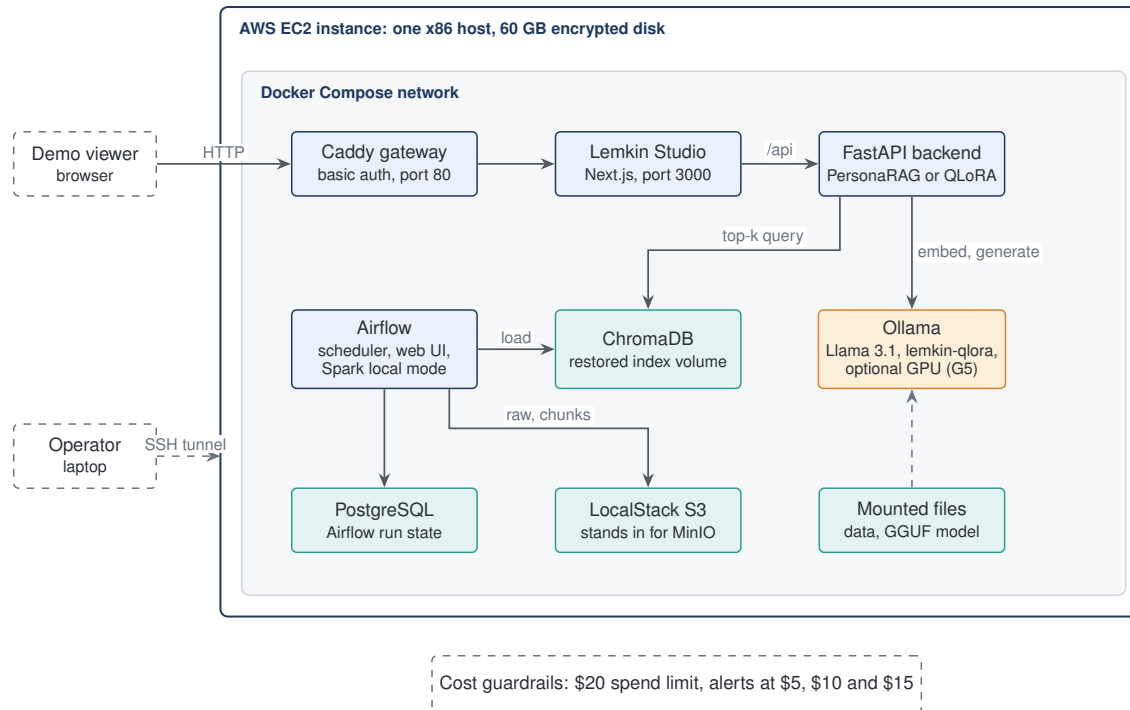


Figure 11. Cloud deployment on one EC2 instance. Users reach the web app through a password gateway or an SSH tunnel. All other services stay on the private Compose network.

integration step.

5.2.2 Supporting platforms, frameworks and cloud environment

We use three environments. The laptop runs the full Compose stack for development. The GPU workstation trains and scores. A single AWS EC2 instance hosts the shared demo (Figure 11).

A few design decisions shaped the cloud setup.

- **One host.** A single instance with a 60 GB encrypted disk keeps cost and complexity low for a demo. A GPU variant for faster generation can be enabled with a second Compose file.
- **Closed by default.** The security group allows SSH only from the operator’s address, and only the web app is exposed, behind HTTP basic authentication. Credentials live in a private environment file that is never committed.
- **S3-compatible storage.** The MinIO images the pipeline used were withdrawn from the public registry, so the EC2 setup uses a LocalStack S3 service under the same service name. No pipeline code had to change.
- **Restore, do not rebuild.** The embedded index is restored from an archive, because re-embedding the whole corpus on a small instance would take hours. A small isolated sample run proves that the ingestion path works without touching the restored index.
- **Cost guardrails.** A \$20 spend limit and alerts at \$5, \$10 and \$15.

5.2.3 Data management solution and database

Figure 12 shows how data moves and where it rests. Raw files are normalized into documents and stored as objects. Spark turns them into chunks with embeddings, which are loaded into

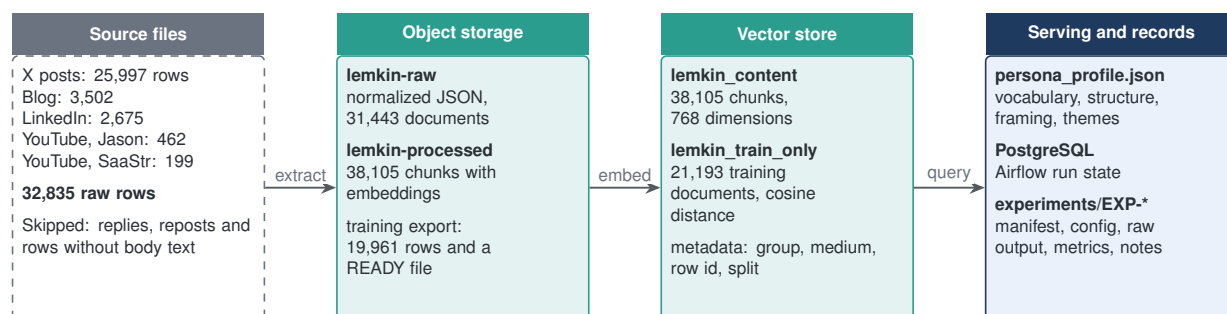


Figure 12. Data stores and the flow between them, with record counts from the full pipeline run.

ChromaDB. A second collection holds only training documents and is the one model E reads. Table 12 lists the main stores, what they hold and why we chose them.

Table 12. Data stores and records.

Store	Record and key fields	Why it is used
Raw JSONL files	One row per post, video or tweet: title, date, URL, text or transcript	Immutable source of truth. Never edited in place.
MinIO or S3 buckets	Normalized document JSON. Chunk records with text, metadata and a 768-dimension vector. Training export and a ready marker	Cheap, file-like storage between pipeline stages. Easy to inspect.
ChromaDB	Chunk text, vector, and metadata: source, group id, medium, row id, split	Vector search with filters. One collection per use, so training-only material is kept apart.
PostgreSQL	Airflow run state and task history	Required by Airflow. Not used for content.
Training dataset	Instruction, empty input, output, medium, source file and line, role (opening, continuation or unchanged)	Fine-tuning input. Every row can be traced to a source line.
Experiment registry	Folder per run with manifest, config, raw output, metrics and notes. The manifest records Git commit, dataset and split hashes, models, and judge	Makes any reported number reproducible.

Data quality and ownership rules are part of the design. Replies, reposts and rows without body text are dropped at ingestion. Spark strips HTML, scrubs boilerplate footers and removes duplicates. Training rows get a promotion filter for event announcements and trailing sign-up lines. Near-duplicates are kept in one split. The index used for generation holds training documents only. Hashes of every dataset and adapter are recorded. Of the 32,835 raw rows, 31,443 documents survived the ingestion filters in the full run.

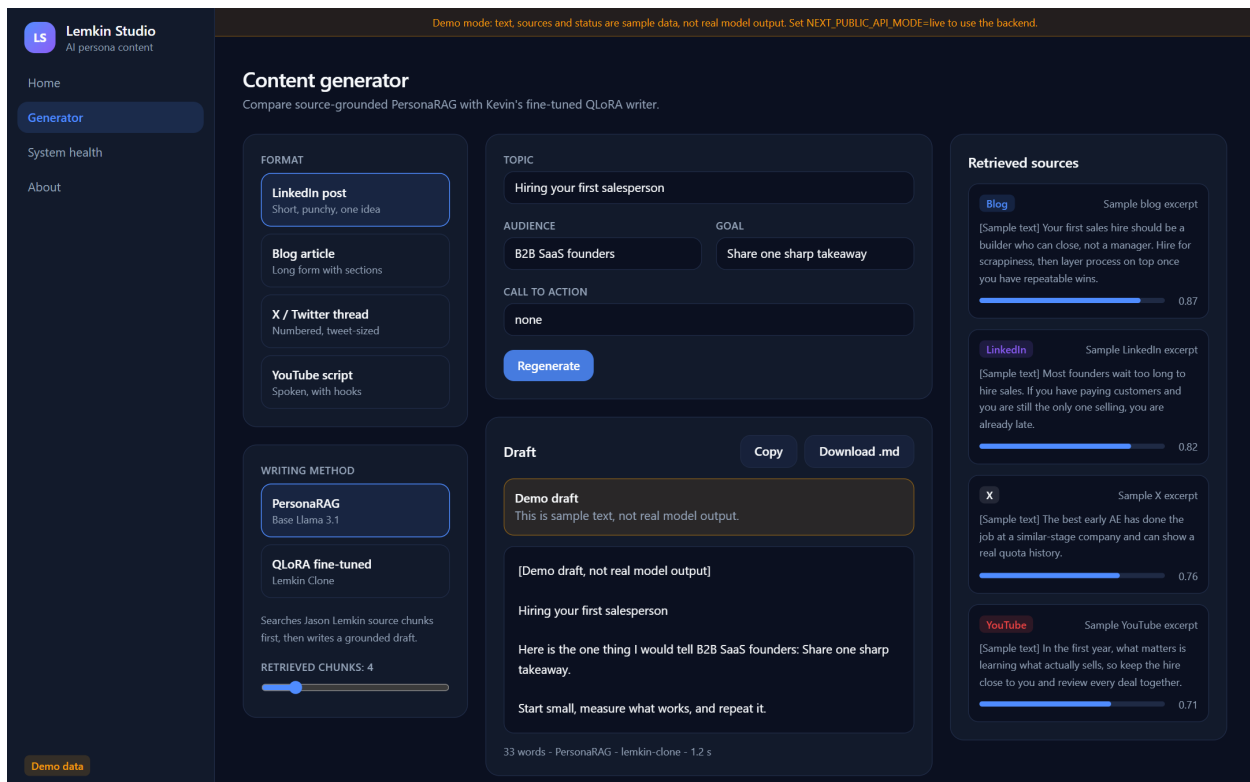


Figure 13. Generator page. The screenshot was taken in the app’s demo mode, so the draft and sources are sample content and not model output.

5.2.4 User interface and data visualization

The interface is a web app we call Lemkin Studio. We designed it for people who write content for a living, so it avoids a chat window and puts the evidence next to the draft. It uses a dark theme with blue and violet accents and rounded cards.

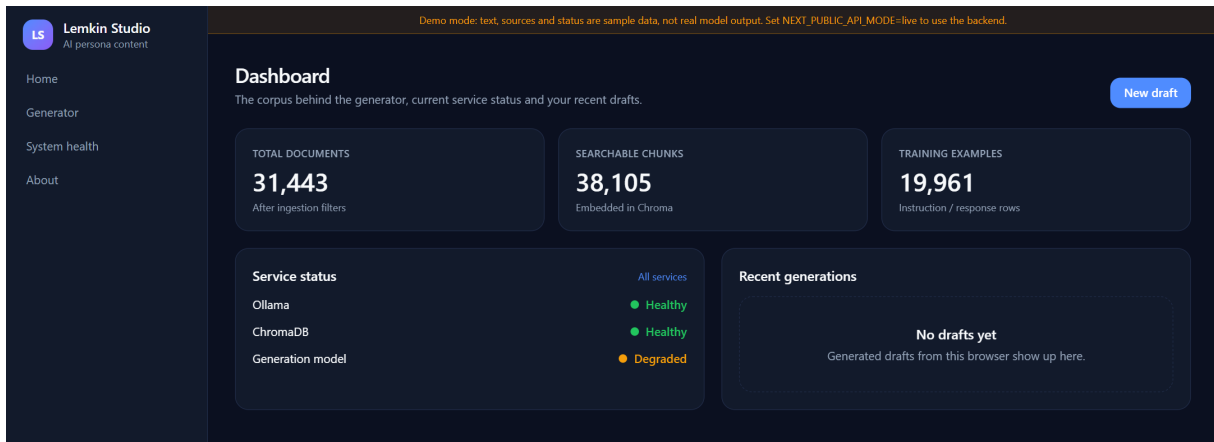
The generator page (Figure 13) has three panels. On the left the user picks the medium and the writing method and sets how many passages to retrieve. The middle panel holds the brief and the draft, with copy and download buttons. On the right, every retrieved passage appears with its source type and a similarity bar, so a reviewer can see what the draft was based on.

The other pages give an overview of the system (Figure 14). The dashboard shows the size of the corpus and the state of the key services. The system-health page lists each service with a green, amber or red marker and a short reason. The evaluation page charts the scores from Section 4.5 and marks any system without results as pending, so empty cells are never filled with invented numbers. Errors from the API are shown with a hint on how to fix them.

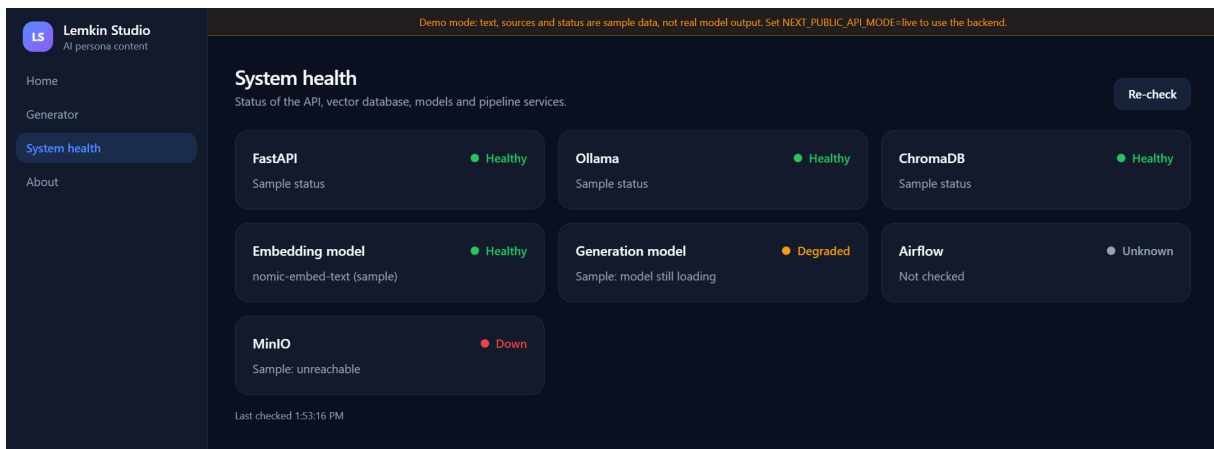
5.3 Intelligent Solution

5.3.1 Solutions for each targeted problem

Table 13 maps each problem to the part of the system that handles it.



(a) Dashboard: corpus size, key service status and recent drafts.



(b) System health: one card per service.

Figure 14. Dashboard and system-health pages, in demo mode.

Table 13. From problem to solution.

Problem	Solution component	How it works
P1 Voice	QLoRA adapter (D, E) and persona profile (B, C)	The adapter learns his phrasing from his posts. The profile gives the base model a written description of his habits.
P2 Substance	Dense retrieval (C, E) and evidence validation (E)	Passages are fetched by meaning. For E, each claim must carry words that appear in its source, or it is dropped.
P3 Medium	Format specifications and medium-matched training rows	Each medium has its own length band and structure. Training rows are labeled by medium.
P4 Completeness	Relabeled training targets (E)	Rows are marked as opening, continuation or whole post, so the model stops learning to end early.
P5 Originality	Copy check with retry (E)	A run of twelve or more words copied from a source triggers one retry.

5.3.2 The integrated solution

Figure 3 shows the integrated path for model E. A content brief is embedded and used to fetch four passages from the training-only index. The base model, with the adapter off, extracts claims and their supporting spans. A deterministic validator keeps the claims whose spans appear in the

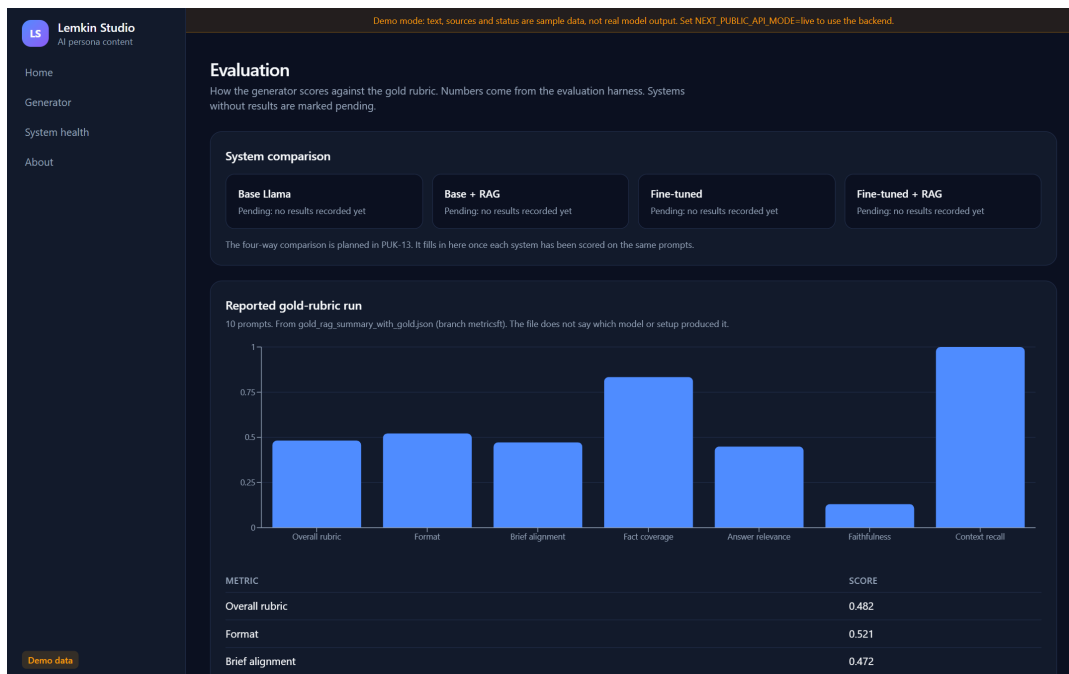


Figure 15. Evaluation page. It charts the committed gold-rubric run and marks the four-system comparison as pending until every system has results.

named source. The adapter is then switched on and writes the draft from the accepted claims. Last, the checks flag unsupported numbers and copied text, and may retry once. The model is loaded once in 4-bit form, and the adapter is toggled between the two steps, so no second copy is needed. The application combines these pieces for the user. The same API accepts a method, so one brief can be sent to PersonaRAG and to QLoRA and the results compared side by side. This is how the models are applied and ensembled. The extraction step, the writing step and the checks form a staged pipeline in which each stage limits what the next one can do.

5.3.3 Input datasets, expected outputs and supporting context

Table 14 lists what goes into the solution and what comes out.

Table 14. Inputs, outputs and context.

Item	Description
Source corpus	Five JSONL files: X posts (25,997 rows), blog (3,502), LinkedIn (2,675), YouTube from Lemkin's channel (462) and from the SaaStr channel (199). In total 32,835 raw rows and 31,443 documents after filtering.
Cleaned corpus and training file	Cleaned source files, then instruction and answer pairs. 42,877 rows after the grouped split, 31,328 after balancing media, 26,545 after relabeling.
Gold panel	30 topics with expected facts and rubric points, each written for four media. 120 prompts.
Persona profile and format specifications	Statistics on vocabulary, structure, framing and themes, and a YAML description of each medium.
Online input	Format, topic, audience, goal, optional call to action, number of passages, and method.
Online output	Draft, retrieved sources, model name, latency, request id and method.
Offline output	Registered experiment folders with traces, per-row scores, summaries and notes.

Table 15. HTTP endpoints of the backend.

Method	Path	Purpose
GET	/health	Overall state, plus a list of services and the reason for any that are down.
GET	/health/dependencies	The same detailed check, used by the system-health page.
GET	/formats	The media the server can write, read from the format specifications.
POST	/generate	Retrieve sources and write a draft with the chosen method.

Item	Description
Supporting context	Ollama for embeddings and generation, ChromaDB for retrieval, the Hugging Face hub for base weights, and a judge model for scoring.

5.3.4 Solution APIs

The web app talks to a FastAPI service. Table 15 lists the endpoints that exist today.

A generation call sends the brief and the method. The values below are illustrative.

Listing 1. Request to /generate.

```
{
  "format": "linkedin_post",
  "topic": "Hiring your first salesperson",
  "audience": "B2B SaaS founders",
  "goal": "Share one sharp takeaway",
  "cta": "none",
  "k": 4,
  "method": "personarag"
}
```

Listing 2. Response from /generate.

```
{
  "draft": "Generated text ...",
  "sources": [
    { "id": "train_176", "text": "Retrieved passage ...", "score": null,
      "source_type": "blog", "title": null, "url": null, "date": null }
  ],
  "model": "llama3.1",
  "latency_ms": 4820,
  "request_id": "a1b2c3",
  "method": "personarag"
}
```

The server validates the request. It returns 400 for an unknown format, 500 if the persona file is missing, and 502 if retrieval or generation fails, with the reason in the message. The Python layer behind it raises typed errors that carry a hint, for example when a model has not been pulled or Chroma is unreachable. The QLoRA method uses a lower temperature and a repeat penalty, and it retries once if the model echoes its instructions back as a draft. The similarity score is not returned yet, so the interface shows “no score reported” for now.

Inside the system, the main programmatic interfaces are the shared pipeline function used by both the command line and the API, the dependency check used by the health endpoints, the Airflow DAG that is triggered to ingest data, and the command-line scripts that train, index and score. Each is documented next to its code.

5.4 System Supporting Environment

Table 16 lists the technologies behind the system, grouped by role.

Table 16. Technologies, platforms and frameworks.

Role	Technology	Used for
Containers	Docker Desktop, Docker Compose	Run the whole stack the same way on a laptop and on EC2
Orchestration	Apache Airflow 2.8.1 on Python 3.10, PostgreSQL	Schedule ingestion, cleaning, loading, persona and training export
Data processing	PySpark in local mode, NLTK	Clean, chunk and embed 31,443 documents
Object storage	MinIO, LocalStack S3 on EC2	Hold raw, processed and training files
Vector database	ChromaDB 1.5.5 with HNSW indexes	Store and search 768-dimension vectors
Model serving	Ollama	Serve <code>nomic-embed-text</code> , Llama 3.1 and the exported fine-tuned model
Models	Llama 3.1 8B Instruct, <code>nomic-embed-text</code> , Qwen 2.5 14B (judge)	Generation, embeddings and scoring
Fine-tuning	Python 3.11, PyTorch 2.10 (CUDA 12.8), Unsloth, TRL, PEFT, bitsandbytes 4-bit	Train and run QLoRA adapters
Export	llama.cpp, GGUF Q4_K_M	Convert merged weights for Ollama
Backend	FastAPI, Pydantic	HTTP API for the web app
Frontend	Next.js 16, React 19, Tailwind CSS 4, Recharts	Web app and evaluation charts
Gateway	Caddy	Password protection for the public demo
Cloud	AWS EC2, optional GPU instance	Shared demo
Quality	pytest (more than 100 tests on the pipeline and API layer), Vitest (12 tests on the web API layer)	Catch regressions
Collaboration	Git and GitHub with one branch and one pull request per task, Linear for issue tracking	Coordinate four people

Hardware. Development and pipeline runs happen on laptops. One of them has 7.8 GB of memory and a 4 GB GPU, which is enough for the embedding model and not for Llama 3.1 8B, so we gave Docker a dedicated 4.8 GB and run the pipeline serially. Training uses a workstation with a 16 GB GPU. The cloud demo uses one x86 instance with 60 GB of encrypted disk.

Practical lessons. Pinning versions mattered more than we expected. A floating Chroma tag broke the connection until the server and the client library were pinned to the same release. The web app and Airflow needed incompatible versions of one library, so the web image installs its own smaller list. MinIO's public images were withdrawn, so we pinned the last release from another registry and later used a compatible service on EC2. These fixes are recorded in the repository so the setup can be repeated.

References

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems* 33, 2020.
- [2] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [3] T. Detrmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems* 36, 2023.
- [4] A. Dubey et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.
- [5] Z. Nussbaum, J. X. Morris, B. Duderstadt and A. Mulyar, “Nomic Embed: Training a reproducible long context text embedder,” arXiv:2402.01613, 2024.
- [6] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, 2020.
- [7] S. Zerhoubi and M. Granitzer, “PersonaRAG: Enhancing retrieval-augmented generation systems with user-centric agents,” arXiv:2407.09394, 2024.
- [8] Z. Guo, L. Xia, Y. Yu, T. Ao and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” arXiv:2410.05779, 2024.
- [9] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.
- [10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *International Conference on Learning Representations*, 2020.
- [11] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems* 36, 2023.